TD SQL bonus : notions avancées

Fonctions SQL, fonctions fenêtres

2026-10-16

L3 MIASHS
Université Paris Cité
Année 2025
Course Homepage
Moodle



> **Documentation**
>
> - pour les fonctions fenêtres : https://www.postgresql.org/docs/current/sql-expressions.html#SYNTAX-WINDOW-FUNCTIONS
> - pour les fonctions SQL : https://www.postgresql.org/docs/current/sql-createfunction.html

Fonctions SQL

Forme générale

```
CREATE OR REPLACE FUNCTION  schema_name.func_name(p_arg1 p_arg1_datatype, ..., [OUT o_arg1 o_arg1_d
RETURNS some type | RECORD | SETOF some type | TABLE (..)
LANGUAGE SQL AS
$$
SQL statement ;
SQL statement ;
...
SQL statement ;
$$ ;
```

> Pour toute la suite du TP, il faut remplacer `username` par votre nom de login.

Fonctions qui retournent un type simple

Ecrire une fonction qui prend en argument une région et renvoie la population maximale parmi les capitales de la région (schéma `world`).

```
CREATE OR REPLACE FUNCTION username.taille_max_capitale_region(p_region text)
RETURNS INTEGER LANGUAGE SQL AS
$$
SELECT MAX(population) AS max_pop
FROM world.country c JOIN world.city cc ON (c.capital=cc.id)
WHERE c.region=p_region AND cc.population IS NOT NULL;
$$ ;
```

Usage : quelle est la population de la capitale la plus peuplée d'Europe de l'Est ? (8389200 correspondant à Moscou)

```
SELECT *
FROM username.taille_max_capitale_region('Eastern Europe');
$$ ;
```

Fonctions qui retournent un type composé

Ecrire une fonction qui prend en argument une région et renvoie le nom et la population de la capitale la plus peuplée de cette région (schéma `world`).

On peut utiliser le *qualifiant* `OUT` pour désigner des paramètres de sortie.

```
CREATE OR REPLACE FUNCTION username.capitale(p_region text, OUT o_name_capital TEXT, OUT o_populati
RETURNS RECORD  LANGUAGE SQL AS
$$
WITH r AS(
  SELECT cc.*, RANK() OVER (PARTITION BY c.region
                            ORDER BY cc.population DESC) AS rnk
  FROM world.country c JOIN world.city cc ON (c.capital=cc.id)
  WHERE c.region=p_region)
SELECT r.name,  r.population
FROM r
WHERE r.rnk = 1 ;
$$ ;
```

Usage : quelle est le nom et la population de la capitale la plus peuplée d'Europe de l'Est ? (Moscow 8389200)

```
SELECT *
FROM username.capitale('Eastern Europe');
```

Fonctions qui retournent un type composé défini par les lignes d'une table

Ecrire une fonction qui prend en argument une région et renvoie la description de la capitale la plus peuplée de cette région (schéma `world`).

```
CREATE OR REPLACE FUNCTION username.capitale(p_region text)
RETURNS world.city  LANGUAGE SQL AS
$$
WITH r AS(
  SELECT cc.*, RANK() OVER (PARTITION BY c.region
                            ORDER BY cc.population DESC) AS rnk
  FROM world.country c JOIN world.city cc ON (c.capital=cc.id)
  WHERE c.region=p_region
)
SELECT r.id, r.name, r.countrycode, r.district, r.population
FROM r
WHERE r.rnk = 1 ;
$$ ;
```

Tester cette fonction.

Fonctions qui retournent une table

Ecrire une fonction qui prend en argument une région et renvoie la table des capitales de la région (schéma `world`).

Table de schéma explicite

On se contente d'abord de renvoyer le nom de la capitale. On explicite le schéma de la table résultat

```
CREATE OR REPLACE FUNCTION username.capitales_region(p_region text)
RETURNS TABLE (name_capital text) LANGUAGE SQL AS
$$
```

```
SELECT cc.name
FROM world.country c JOIN world.city cc ON (c.capital=cc.id)
WHERE c.region=p_region ;
$$ ;
```

Tester cette fonction.

Table de même schéma qu'une autre table

Documentation

On veut maintenant récupérer une table de même schéma que `city`.

> La solution est très simple. On profite de ce qu'à chaque table correspond un type de même nom et on utilise le mot-clé `SETOF`.

```
CREATE OR REPLACE FUNCTION username.capitales_region_large(p_region text)
RETURNS SETOF world.city  LANGUAGE SQL AS
$$
SELECT cc.*
FROM world.country c JOIN world.city cc ON (c.capital=cc.id)
WHERE c.region=p_region ;
$$ ;
```

Tester cette fonction.

Questions

1. (schéma `pagila`) Dans votre schéma personnel `username` (il faut remplacer `username` par votre nom de login), écrire une fonction SQL `film_id_cat` qui prend en paramètre une chaîne de caractère `s` et renvoie la liste des films de catégorie `s`.

> **Solution**
>
> ```
> CREATE OR REPLACE FUNCTION
>   username.film_id_cat(s text)
> RETURNS TABLE(film_id smallint) AS
> $$
>     SELECT fc.film_id
>     FROM
>         film_category fc
>   JOIN
>         category ca
>   ON (fc.category_id=ca.category_id)
>     WHERE
>     ca.name=s ;
> $$ LANGUAGE sql ;
> ```

2. (schéma `pagila`) Avec la fonction précédente, lister les acteurs qui ont déjà joué dans un film d'horreur (catégorie 'Horror').
3. (schéma `pagila`) Avec la fonction précédente, lister les acteurs qui n'ont jamais joué dans une comédie (`Comedy`) . (53 lignes)

☢ **Attention ! Cette requête ne répond pas à la question :**

```
SELECT DISTINCT ac.*
FROM actor ac NATURAL JOIN
     (SELECT * FROM film_actor
      WHERE film_id NOT IN
        (SELECT * FROM film_id_cat('Comedy') )
      ) as X;
```

Elle répond à la question : *Quels sont les acteurs qui ont joué dans un film qui n'est pas une comédie ?*

Nous pouvons par exemple d'abord calculer la liste des `actor_id` des acteurs qui ont joué dans au moins une comédie, puis calculer la liste des `actor_id` des acteurs présents dans la base et faire la différence.

4. (schéma `pagila`) Avec la fonction précédente, lister les acteurs qui ont joué dans un film d'horreur ('Horror') et dans un film pour enfant ('Children'). (129 lignes)

Ici l'erreur la plus fréquente consiste à écrire

```
SELECT
  actor_id
FROM
  film_actor AS fa
WHERE
  fa.film_id IN (
    SELECT *
    FROM entid.film_id_cat('Children')
  ) AND
  fa.film_id IN (
    SELECT *
    FROM entid.film_id_cat('Horror')
  );
```

Le résultat est vide et la requête ne correspond pas à la question posée.
Elle calcule les `actor_id` des acteurs qui ont joué dans au moins un film qui relève simultanément des catégories `Horror` et `Children` (ce genre de film est assez rare).
Pour calculer un résultat correct, il faut pour chaque valeur a de `actor_id` rechercher deux tuples (pas nécessairement distincts) de `film_actor`, où l'attribut `actor_id` vaut a, et tel que, dans un cas, `film_id` désigne un film pour enfants et, dans l'autre, un film d'horreur.
En algèbre relationnelle, cela donne par exemple :

$$\pi_{\texttt{last_name},\texttt{first_name}}\Big(\texttt{actor} \bowtie \Big(\pi_{\texttt{actor_id}}(\texttt{film_actor} \bowtie \texttt{film_id_cat('Children')}) \bigcap \pi_{\texttt{actor_id}}(\texttt{film_actor} \bowtie \texttt{film_id_cat('Horror')})\Big)\Big)$$

5. (schéma `pagila`) Ecrire une fonction `actor_category(p_nom text, p_prenom text)` qui prend en argument le nom et le prénom d'un acteur (d'une actrice) et renvoie la liste des noms des catégories de films dans lesquels il/elle a joué.

6. (schéma `pagila`) Ecrire une fonction `co_actors(p_nom text, p_prenom text)` qui renvoie les noms et prénoms des acteurs qui jouent dans un film où apparaît un acteur ou une actrice dont le nom et le prénom sont donnés en argument.

7. (schéma `world`) Écrire une fonction `plus_peuplee(p_countrycode text)` qui, étant donné le code d'un pays, renvoie le nom de la ville la plus peuplée de ce pays.

8. (schéma `world`) Écrire une fonction `langues_region(p_continent text)` qui étant donné le nom d'un continent, renvoie le nombre moyen de langues parlées par pays dans chaque région. L'entête de cette fonction doit être :

```
FUNCTION langues_region(p_continent TEXT)
RETURNS TABLE(region TEXT, nbmoy NUMERIC)
```

9. (schéma `world`) Écrire une requête qui liste trois attributs, pour chaque pays où on parle français, ordonné par population croissante :

- `name_country`,
- `cumul_loc` qui donne le nombre cumulé de locuteurs du français dans les pays où on parle français, dans l'ordre croissant des populations des pays,
- `cum_pop` qui donne la population cumulée des pays où on parle français, dans l'ordre croissant des populations des pays.

Utilisez une fonction fenêtre (`WINDOW`) sans partition.

Pour trouver les pays où on parle français, utilisez l'expression `language like '%French%'`. Vous remarquerez que dans certains pays, il existe plusieurs variétés de 'French'. Veillez à compter les locuteurs de toutes les formes du Français.
Que se passe-t-il si un locuteur parle plusieurs formes de Français ?

Schéma `babynames`

Chaque tuple de la table `bebes` indique le `nombre` de nouveaux-nés de genre `sexe`, prénommés `prenom` mis au monde durant l'année `annee` en France.

10. (schéma `babynames`) Pour chaque année, chaque sexe, donner les dix prénoms les plus populaires et leur rang de popularité.
11. (schéma `babynames`) Créer dans votre schéma personnel une vue `top_10` qui, interrogée, renvoie le résultat de la requête précédente
12. (schéma `babynames`) Écrire une fonction qui prend en argument un prénom et renvoie une table de schéma `(annee, sexe, rang)` et indique le rang du prénom pour chaque année et chaque sexe.
13. (schéma `babynames`) Pour chaque année et sexe, calculez le nombre total de naissances.
14. (schéma `babynames`) Créer dans votre schéma personnel une vue `naissances_sexe_annee` qui, interrogée, renvoie le résultat de la requête précédente
15. (schéma `babynames`) Pour chaque année, calculer le rapport entre le nombre total de naissances féminines et nombre total de naissances masculines.

On effectue avec les outils SQL une opération qui s'appelle un *pivot en forme élargie*.

16. (schéma `babynames`) Créer dans votre schéma personnel une vue `naissances_f_h_annee` qui, interrogée, renvoie le résultat de la requête précédente.
17. (schéma `babynames`) Pour chaque année, calculer le rapport entre le nombre total de naissances féminines et le nombre total de naissances.
18. (schéma `babynames`) Créer dans votre schéma personnel une vue `naissances_sexe_annee` qui, interrogée, renvoie le résultat de la requête précédente.
19. (schéma `babynames`) Écrire une fonction qui prend en argument une année `yyyy` et renvoie une table de schéma `(annee, sexe, prenom, prop)` où `prenom` est un des dix prénoms les plus populaires donné en l'aneee `yyyy` et calculer la proportion de nouveaux-nés portant ce prénom année par année.



Tenez compte du fait que certains noms peuvent avoir une occurrence nulle certaines années.